

CUAOA: A Novel CUDA-Accelerated Simulation Framework for the QAOA

Jonas Stein[✉]
LMU Munich, Germany
Aqarios GmbH, Germany
jonas.stein@ifi.lmu.de

Jonas Blenninger[✉]
LMU Munich, Germany
Aqarios GmbH, Germany
jonas.blenninger@aqarios.com

David Bucher[✉]
Aqarios GmbH, Germany
david.bucher@aqarios.com

Peter J. Eder[✉]
Siemens AG, Germany
peter-josef.eder@siemens.com

Elif Çetiner
LMU Munich, Germany
elif.cetiner@tum.de

Maximilian Zorn[✉]
LMU Munich, Germany
maximilian.zorn@ifi.lmu.de

Claudia Linnhoff-Popien[✉]
LMU Munich, Germany
linnhoff@ifi.lmu.de

Abstract—The Quantum Approximate Optimization Algorithm (QAOA) is a prominent quantum algorithm designed to find approximate solutions to combinatorial optimization problems, which are challenging for classical computers. In the current era, where quantum hardware is constrained by noise and limited qubit availability, simulating the QAOA remains essential for research. However, existing state-of-the-art simulation frameworks suffer from long execution times or lack comprehensive functionality, usability, and versatility, often requiring users to implement essential features themselves. Additionally, these frameworks are primarily restricted to Python, limiting their use in safer and faster languages like Rust, which offer, e.g., advanced parallelization capabilities. In this paper, we develop a GPU accelerated QAOA simulation framework utilizing the NVIDIA CUDA toolkit. This framework offers a complete interface for QAOA simulations, enabling the calculation of (exact) expectation values, direct access to the statevector, fast sampling, and high-performance optimization methods using an advanced state-of-the-art gradient calculation technique. The framework is designed for use in Python and Rust, providing flexibility for integration into a wide range of applications, including those requiring fast algorithm implementations leveraging QAOA at its core. The new framework's performance is rigorously benchmarked on the MaxCut problem and compared against the current state-of-the-art general-purpose quantum circuit simulation frameworks Qiskit and Pennylane as well as the specialized QAOA simulation tool QOKit. Our evaluation shows that our approach outperforms the existing state-of-the-art solutions in terms of runtime up to multiple orders of magnitude. Our implementation is publicly available at <https://github.com/JFLXB/cuaoa> and Zenodo [1].

Index Terms—Quantum Computing, Quantum Optimization, QAOA, Quantum Circuit Simulation, CUDA, HPC

I. INTRODUCTION

One of the most promising approaches for quantum advantage in the domain of optimization problems is the Quantum Approximate Optimization Algorithm (QAOA) [2]. As a parameterized version of the Quantum Adiabatic Algorithm [3], the QAOA utilizes a quantum phenomenon, i.e., the Adiabatic Theorem [4], to solve optimization problems approximately. In consequence of many results about improved scaling performance of the QAOA for important optimization problems compared to classical state-of-the-art solvers [5]–[7], much scientific effort goes into achieving large scale

numerical simulations to explore possible quantum advantages empirically [8]–[10].

Beyond tensor networks, decision diagrams, and other approaches, the current standard for large-scale, noise-free quantum circuit simulations relies on efficient matrix multiplication via GPUs and more specifically the cuStateVec SDK [11], which allows for a translation of circuit instructions written in Qiskit [12], Pennylane [13], or other SDKs, to CUDA, the toolkit for instructing computations on NVIDIA GPUs [14].

Due to its specific quantum circuit structure, the runtime complexity of simulating a standard p -depth, n -qubit, X -mixer-based, QAOA circuit is $\mathcal{O}(pn2^n)$ instead of the $\mathcal{O}(p4^n)$ for general p -depth quantum circuits [15]. As we expect no further speedup for the simulation of the X -mixer (cf. [15, 16]), and to allow for arbitrary mixers (cf. [17]), we focus on speedup gains based on the diagonal structure of the cost operator (cf. [15]).

In the current state-of-the-art QAOA-simulator QOKit [15], Lykov et al. exploit this diagonal structure of the cost operator by precomputing the costs for all possible solutions in parallel and then using these for the cost unitary application as well as the expectation value calculation. However, QOKit is written in Python and hence cannot use CUDA natively, hindering itself from achieving the fastest possible computation. What is more, the two key features besides the precomputation of the cost Hamiltonian are not executable in its provided code due to missing implementation: the cuStateVec simulator and the gradient computation [18].

Resolving the shortcomings of QOKit, we develop a CUDA-based QAOA simulator (CUAOA) that is optimized to execute all practically relevant operations for QAOA-evaluation in native CUDA, oriented towards a single-GPU setting. To ensure compatibility with the expected end-user programming language Python (e.g., via PyO3 [19]) while allowing for direct CUDA access, we employ Rust with integrated C/C++ modules (via Foreign Function Interface (FFI)). Analog to the current state of the art, we use cuStateVec for all computations that do not concern the cost unitary, e.g., the mixer application and the sampling process. Allowing for significant speedups

compared to current state-of-the-art QAOA simulators, we propose CUDA-native implementations of

- the precomputation of the cost Hamiltonian,
- the application of the cost unitary,
- the calculation of the expectation value, and
- the gradient computation.

The remainder of this paper is structured as follows. In Sec. II, we outline preliminaries on the simulation of the QAOA, the adjoint differentiation method, and CUDA. In Sec. IV, CUAOA is presented and subsequently evaluated in Sec. V. Finally, we conclude our findings in Sec. VI.

II. BACKGROUND

In this section, we provide preliminaries on the QAOA, a method to efficiently compute gradients in classically simulated quantum circuits and CUDA.

A. The Quantum Approximate Optimization Algorithm

Given a combinatorial optimization problem by an objective function $f: \{0,1\}^n \rightarrow \mathbb{R}$, the QAOA conducts the following steps to approximate the optimal solution [2]:

- 1) Mapping the objective values onto the eigenvalues of the diagonal cost Hamiltonian $H_C = \sum_x f(x) |x\rangle\langle x|$.
- 2) Preparing a system in the ground state of the mixer Hamiltonian, i.e., usually $|+\rangle^{\otimes n}$ for $H_M = -\sum_{i=1}^n \sigma_i^x$.
- 3) Simulating the time evolution $\exp\left(i \int_0^T H_s(t) dt\right)$ approximately, where $H_s(t) = (1-s(t))H_M + s(t)H_C$ governs the adiabatic evolution and a bijective $s: [0, T] \rightarrow [0, 1]$ increases monotonically for given $T > 0$.
- 4) Measuring the resulting state and remapping it to its corresponding solution of the objective function f .

To simulate the time evolution of H_s in a quantum circuit, a discretization into $p \in \mathbb{N}$ Hamiltonians $H_s(1/T), \dots, H_s(T)$ via Trotterization is carried out, amounting the following unitary operation forming the QAOA:

$$U(\beta, \gamma) = U_M(\beta_p)U_C(\gamma_p) \dots U_M(\beta_1)U_C(\gamma_1)H^{\otimes n}, \quad (1)$$

where β_i and γ_i control the speed of the time evolution and $U_M(\beta_i) = e^{-i\beta_i H_M}$, $U_C(\gamma_i) = e^{-i\gamma_i H_C}$, s.t. $U(\beta, \gamma)$ approaches adiabatic time evolution for $p \rightarrow \infty$, and constant speed, i.e., $\beta_i = 1 - i/p$, and $\gamma_i = i/p$ [20].

B. Adjoint Differentiation

The adjoint differentiation method [21] exploits the possibility to clone statevectors in classical quantum circuit simulators to yield a runtime of $\mathcal{O}(P)$ instead of the generally employed parameter shift rule which has complexity $\mathcal{O}(P \cdot m)$ [22], where P is the number of possibly parameterized layers in the quantum circuit and m is the number of parameters. These complexities state the query complexity of simulating the application of a layer of gates, which is generally $\mathcal{O}(4^n)$ for an n -qubit quantum register.

In the following we assume that each circuit layer U_i has exactly one parameter θ_i , which is the case for the standard form of QAOA. For details on adaptations necessary for this

approach to work for more general circuit layers involving repeated parameters and multiple parameters per layer see Ref. [21, Appendix B]. Notably, these increase the runtime complexity by constant factors.

The adjoint differentiation method exploits the hermiticity of the partial derivative of the measurement operator M , i.e.,

$$\begin{aligned} \frac{\partial \langle M \rangle}{\partial \theta_i} &= \langle 0 | U_1^\dagger \dots \frac{\partial U_i^\dagger}{\partial \theta_i} \dots U_P^\dagger M U_P \dots U_i \dots U_1 | 0 \rangle \\ &\quad + \langle 0 | U_1^\dagger \dots U_i^\dagger \dots U_P^\dagger M U_P \dots \frac{\partial U_i}{\partial \theta_i} \dots U_1 | 0 \rangle \\ &= 2\Re \left(\langle 0 | U_1^\dagger \dots U_i^\dagger \dots U_P^\dagger M U_P \dots \frac{\partial U_i}{\partial \theta_i} \dots U_1 | 0 \rangle \right), \end{aligned}$$

which can be written as $\nabla_{\theta_i} \langle M \rangle = 2\Re(\langle b_i | \partial U_i / \partial \theta_i | k_i \rangle)$, where $\langle b_i | := \langle 0 | U_1^\dagger \dots U_i^\dagger \dots U_P^\dagger M U_P \dots U_{i+1}$ and $| k_i \rangle := U_{i-1} \dots U_1 | 0 \rangle$ can be computed recursively via $\langle b_{i+1} | = \langle b_i | U_{i+1}^\dagger$ and $| k_{i+1} \rangle = U_i | k_i \rangle$. Due to this recursive nature, it takes $\mathcal{O}(P)$ layer executions to calculate $\nabla_{\theta_1} \langle M \rangle$ and then $\mathcal{O}(1)$ layer executions for all other partial derivatives yielding the stated overall runtime complexity of $\mathcal{O}(P)$.

C. CUDA

The Compute Unified Device Architecture (CUDA) is a parallel computing platform and application programming interface (API) model that enables developers to directly access NVIDIA Graphics Processing Units (GPUs) [14]. The main advantage of GPUs over CPUs lies in the ability to execute computations in a massively parallelized manner. This is especially relevant for applications like matrix multiplication, which can be divided into many smaller, independent computations.

CUDA is an extension of the C/C++ programming languages that has a hierarchical structure centered around threads. Calls from the CPU to CUDA are done via kernel functions, known as kernels, that run on the GPU. These kernels are executed by a grid of thread blocks, with each block containing multiple threads. This hierarchical arrangement enables efficient use of the GPU's resources and provides precise control over each thread's behavior. Threads within the same block can share data through shared memory, which is unique to each block and generally faster than global memory. Communication between threads of different blocks occurs through global memory. [23]

III. RELATED WORK

While multiple frameworks exist that are aimed at providing QAOA circuit implementations of various versions of the QAOA (e.g., OpenQAOA [24] and JuliaQAOA [25]), only QOKit [15] is aimed at—and capable of—achieving a significant speedup through GPU usage. Therefore we focus on QOKit in the remainder of this section.

QOKit is targeted towards simulating the QAOA involving large amounts of qubits and offers a multi-GPU approach that shares information about the statevector via OpenMPI [26]. The two key features of QOKit are a parallelized computation

and application of the cost operator and an algorithm to apply the X -mixer in time $\mathcal{O}(n2^n)$.

At the time of this paper being published, QOKit is limited by significant shortcomings in their published code, i.e., missing support of the cuStateVec circuit simulator as well as missing gradient calculation, which both are key components for achieving the shortest runtimes possible. Furthermore, QOKit is limited by its implementation being carried out in Python, which manifests in that their proposed mixer unitary, as well as their OpenMPI-based parallelization perform worse compared to plain cuStateVec [15]. Also, for extracting information about the statevector, e.g., for sampling, the probabilities of the statevector are copied to the CPU, which displays a significant runtime bottleneck.

Based on the results presented in Ref. [15], the only clear speedup that QOKit provides over a plain cuStateVec-based implementation appears to be the efficient precomputation of the cost operator and its application in diagonal form. The precomputation is based on the polynomial representation of the objective function

$$f(s) = \sum_{k=1}^L w_k \prod_{i \in t_k} s_i, \quad (2)$$

where $s \in \{-1, 1\}^n$ and $\mathcal{T} := \{(w_1, t_1), \dots, (w_L, t_L)\}$ defines the polynomial terms through the indices of the involved variables $t_k \subseteq \{i \mid 1 \leq i \leq n\}$ and their associated weight $w_k \in \mathbb{R}$. For the computation of $f(s)$ of all possible inputs s , an array of zeros is allocated on the GPU and then a GPU kernel iterating over all terms in $\mathcal{T}$ is applied in parallel for each entry in the array. The value of each term is calculated using bitwise-XOR and population count operations to determine the sign of $\prod_{i \in t_k} s_i$. The application of the cost unitary is executed by an element wise product of the statevector with $\exp(-i\gamma_i f(s))$.

IV. METHODOLOGY

Aiming to exploit the diagonal structure of the cost operator in the QAOA, we now propose CUAOA, a CUDA-accelerated, single-GPU quantum circuit simulator for the QAOA that annihilates the shortcomings of QOKit. To offer the same convenience to end-users as QOKit—a Python interface—while enabling seamless CUDA-operability, the core module of CUAOA is written in Rust. This allows access to CUDA-instructions written in C/C++ via FFI and the integration from within Python via libraries like PyO3 [19]. This also yields the significant advantage of running the exact same program instructions much faster compared to a Python implementation, just because Rust is a compiled language, i.e., the code is compiled directly to machine code before execution.

Starting from a baseline of the state of the art in circuit-agnostic noise-free quantum circuit simulation, i.e., cuStateVec, for our QAOA simulation, we now show how every circuit simulation component that involves a cost operator can be implemented more efficiently via CUDA-based implementation of the operator in its diagonal form.

A. Cost function representation

In standard QAOA simulation, the cost operator is represented by quantum gates modelling the cost function. However, as the cost operator is diagonal in most practical applications (which is a result of basis encoding), the application of the cost operator in a classical circuit simulation can be carried out directly through a multiplication with a diagonal matrix, which can be fully parallelized on GPUs.

Improving on the cost function computation of QOKit [15], we work with a 0–1 based function representation that reduces the number of necessary additions significantly: Encode the indices of the n binary variables using one-hot encoding, we can represent a polynomial objective function as

$$f(x) = \sum_{k=1}^L w_k \left(x \Leftrightarrow x \wedge_b \left[\sum_{v_i \in t_k} 2^i \right]_2 \right), \quad (3)$$

where each term t_k consists of variable indices $v_i \in [n]$ and $\Leftrightarrow$ denotes the logical equivalency that yields 1 or 0 respectively. As the binary string $[\sum_{v_i \in t_k} 2^i]_2$ can easily be computed through bitwise logical AND operations (denoted $\wedge_b$) of the one-hot representation of each term t_k , and as only the entries for which the bitstring x is nonzero have to be considered for any given x , a large amount of terms can be ignored in computation. This shortcut is not exploited in QOKit, which iterates over all terms. Thus, our approach can be significantly faster for polynomials that have a large amount of small degree terms, which even are the norm in practice. For problems inheriting symmetries (e.g., the MaxCut problem), many cost values equal each other, such that additional speedups could be gained. However, we refrain from such optimizations to maintain problem agnosticity.

B. QAOA circuit simulation

The CUAOA starts by allocating memory for the statevector as well as the cost Hamiltonian and stores the pointers referencing the respective GPU memory. In addition, a CUDA stream is created and its reference is stored in the handle, which allows multiple kernels associated with different streams to be executed in parallel on the same GPU. Further, the handle for interactions with the cuStateVec library is initialized with the handle's stream and subsequently stored. Memory for other variables is not allocated upon the handle's initialization, but only later when they are actually needed, to reduce memory usage.

The statevector is initialized as an array of the CUDA double type for complex numbers `cuDoubleComplex` in parallel for each entry. As all evaluations are carried out for the standard form of QAOA with an X -Mixer, we directly initialize all values of the array to $1/\sqrt{2^n}$. The cost Hamiltonian is initialized as an array of double precision entries and the value of each entry is computed via Eq. (3) in parallel. The cost unitary is applied to the statevector $|\psi\rangle$ exploiting Eulers formula $\exp(i\theta) = \cos \theta + i \sin \theta$ through $\psi_i \mapsto [\cos(-\gamma_i f(x)) + i \sin(-\gamma_i f(x))] \cdot \psi_i$. The variational parameter γ_i for this is passed as a double-precision input

to this operation and the CUDA built-in function for multiplication `cuCmul` is used. Note that while `cuStateVec` also offers a function to directly apply a diagonal matrix¹, they do not support the in-place multiplication of the γ_i , which would have to be done through another kernel leading to resource inefficiency, in turn, motivating the stated approach.

To apply the mixer unitary, `custatevecApplyMatrix` is used. As we only used the X -mixer in our evaluation, this reduces to the application of an $R_x(-2\beta_i)$ gate for every qubit.

C. Gradient computation

To compute the gradient of a QAOA circuit with respect to its variational parameters γ and β , we employ the adjoint differentiation technique outlined in Sec. II-B, as it is the state of the art for gradient calculation in classical circuit simulation. For the QAOA, another simplification in gradient calculation of each layer arises from the well-known identity $\frac{\partial}{\partial t} e^{tA} = A e^{tA}$, which implies that $\frac{\partial}{\partial t} e^{-i\gamma_i H_C} = -i H_C e^{-i\gamma_i H_C}$ and $\frac{\partial}{\partial t} e^{-i\gamma_i H_M} = -i H_M e^{-i\gamma_i H_M}$. While the application of H_C is trivial, the application of H_M reduces to a layer of X -gates for all of our evaluation runs, as we only consider the standard X -mixer. The uncomputation needed for each gradient calculation step described in Sec. II-B also simplifies based on $\frac{\partial}{\partial t} e^{tA} = A e^{tA}$, as only $-i H_M$ and $-i H_C$ respectively have to be uncomputed. As both operators are Hermitian (i.e., $H_M = H_M^\dagger$ and $H_C = H_C^\dagger$), this uncomputation can be done by applying $i H_M$ and $i H_C$ respectively. In our implementation we get rid of the introduced imaginary number i by switching from the real to the imaginary part (cf. Sec. II-B).

D. Retrieving Results from the GPU

Arguably the most important output of a QAOA simulator is the expectation value $\langle \psi | H_C | \psi \rangle = \sum_{i=1}^{2^n} f(x_i) |\psi_i|^2$. To compute this sum, we calculate $f(x_i) |\psi_i|^2$ for all i based on the resulting QAOA statevector $|\psi\rangle$ and the cost operator, and store the result in a new array of doubles. Then we calculate the sum of all components of this array by braking it down into a tree-like hierarchical structure where, at each level, always two elements are added in parallel, amounting to a total of $\log_2(2^n)$ sequential computations.

To sample from the statevector, we use the sampling functionality offered in `cuStateVec`. This has the big advantage that the statevector does not have to be copied to the CPU thus evading any memory-transformation bottlenecks. In addition to the sampled solution bitstring, we also output the respective objective value, as it is stored on the GPU anyway and thus save additional computational efforts for the user. What is more, our implementation also supports the edge-case of exporting the complete statevector from GPU to CPU.

V. EVALUATION

To evaluate the performance of CUAOA, we examine its runtime for the full circuit execution with regards to outputting the expected value as well as sampling, and also its performance in parameter training using gradient-based methods.

¹Namely `custatevecApplyGeneralizedPermutationMatrix`.

In alignment with QOKit's evaluation [15], we consider the MaxCut problem with three types of graphs ranging from 6 to 29 vertices: (1) random graphs generated based off the Erdős-Rényi $G(n, p)$ model [27] with 25%, 50%, and 75% connectivity, (2) random 3-regular graphs, and (3), complete graphs. Generating five instances per vertex-count and graph type (considering each Erdős-Rényi-connectivity as its own type), this results in a dataset of 444 graphs. Further, all experiments are run with a QAOA depth of $p = 6$, to allow for better comparability with the original results of Ref. [15]. Additional test runs (not shown here), reveal the theoretically expected linear scaling in the runtime of the QAOA simulation for increasing p .

As baselines, we employ the current state-of-the-art HPC QAOA simulator QOKit as well as standard QAOA implementations in Qiskit and PennyLane. For Qiskit and PennyLane, `cuStateVec` is used to run the experiments on a GPU. The circuit simulation for QOKit is based on numba (which translates Python code into machine code upon compilation using Just-in-Time compilation and can natively be run on GPUs [28]), as the `cuStateVec` variant of QOKit is not implemented in the currently available version of their code. While this limits the comparability of our results to the results published for QOKit, our methodology shows that all of our modifications to the QAOA simulation yield theory-proven improvements over QOKit, even when `cuStateVec` was executable for QOKit.

All experiments are run on a high-end consumer-grade system running EndeavourOS Linux x86_64 with Linux Kernel version 6.8.7-arch1-1, 64GB of RAM, an AMD Ryzen 7 3700X CPU (16 cores @ 3.600 GHz), and an NVIDIA GeForce RTX 3090 GPU. All executions are started from within a Python script.

A. Runtime of a single QAOA circuit simulation

To examine the runtime of a plain, noise-free QAOA circuit execution closing with a measurement of the expectation value, we compare CUAOA with all three baselines (QOKit, Qiskit, and PennyLane) in Fig. 1. For PennyLane, a memory allocation error occurred for problem instances exceeding 26 vertices, resulting in only 391 graphs being run successfully. Aside from this technical detail, we can observe that CUAOA performs best for all runs, even outperforming the state-of-the-art baselines by orders of magnitude in all small to medium-sized problem instances. It becomes evident that the effect of exponential runtime scaling only starts to manifest at around 16 qubits for CUAOA.

In-line with theoretical considerations, the application of the mixer unitary is eventually the biggest bottleneck for the runtime, as can be seen by comparing Fig. 1 with the results from sampling-based QAOA runs displayed in Fig. 2, where the slope of the runtimes of all simulators become identical for problems above 20 qubits. The main reason that prevents further speedups is the sequential application of all mixer gates. Since every single mixer gate application modifies the memory of the entire statevector, parallelization is impossible.

Finally, the fact that the runtime of CUAOA is up to an order of magnitude better than QOKit's for problem instances below 20 qubits is necessarily the consequence of our CUDA-native implementation, as well as the optimized computations we introduced in Sec. IV. For problem instances beyond 20 qubits, this reduced overhead apparently marginalizes, leading to roughly equal runtimes, but with CUAOA still outperforming QOKit.

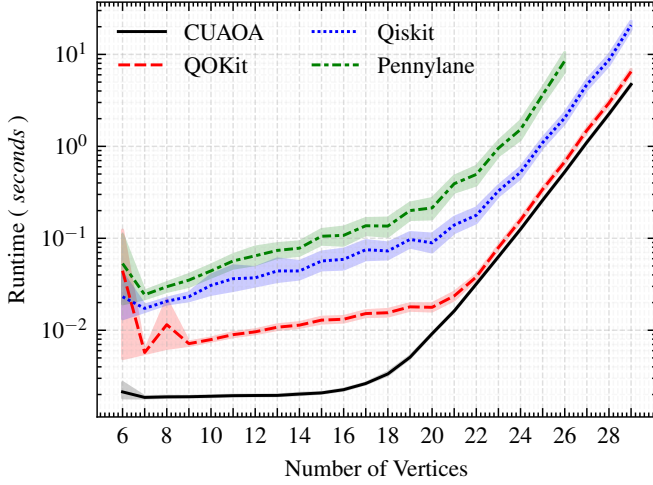


Fig. 1: Runtime wrt. expectation value for QAOA with $p = 6$.

To evaluate the runtime of sampling bitstrings from the resulting statevector of the QAOA, Fig. 2 displays the results for a QAOA run with $p = 6$ and 1024 shots. To allow for better comparison to QOKit, whose available implementation at the time this article is published does not support sampling, we implemented a minimal-effort Python script. For this, we utilize QOKit's functionality to extract the probabilities of the statevector to the CPU. Subsequently, we use the standard Python random number generator to sample from the array containing the associated cumulative probabilities (which was computed using the `cumsum` function of `numpy`).

The results of CUAOA mirror those of the expectation value, showing the high degree of efficiency of both, i.e., not exceeding the runtime of the mixer application. While Qiskit performs quite well, PennyLane is significantly worse compared to the runtimes of the expectation value. The reasons for this are somewhat unclear, but indicate different implementations for sampling, especially for smaller circuit sizes. As expected, our CPU-based implementation of sampling for QOKit is hardly competitive as soon as the dimensionality of the statevector increases.

B. Parameter Training

Examining gradient-based parameter training, we now study the runtime of gradient computation for all parameters. As neither QOKit nor the `cuStateVec`-based Qiskit implementation allows for a computation of the gradient in the form of their available implementation at the time this paper is published, our experiment is restricted to a comparison of PennyLane and

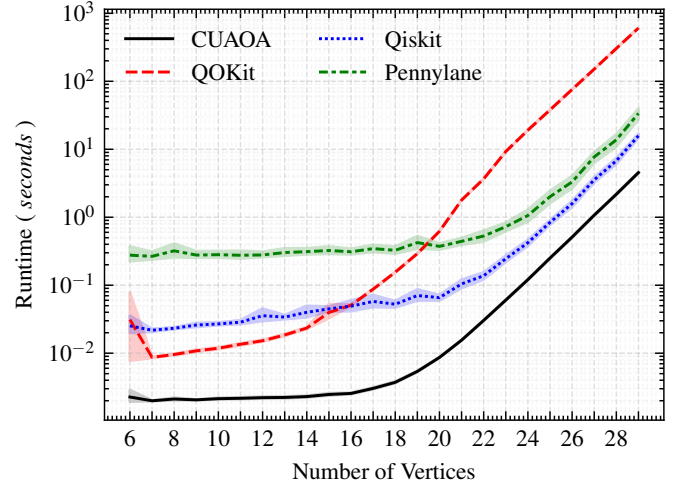


Fig. 2: Runtime for sampling with 1024 shots and $p = 6$.

CUAOA. As the PennyLane implementation also implements the adjoint method, this is a fair comparison to our QAOA-aware enhanced version of the adjoint method. Analog to earlier evaluation runs, PennyLane again fails to execute graphs beyond 26 vertices, resulting in only 391 successfully executed problem instances. Fig. 3 clearly shows that CUAOA outperforms PennyLane by multiple orders of magnitude for problem instances up to 18 qubits, being roughly 100 times faster in almost all runs. For a larger number of qubits, this gap closes to about a 10-fold speedup, with a significant runtime increase at around 18 qubits analog to what has manifested in the plain circuit evaluation in Sec. V-A.

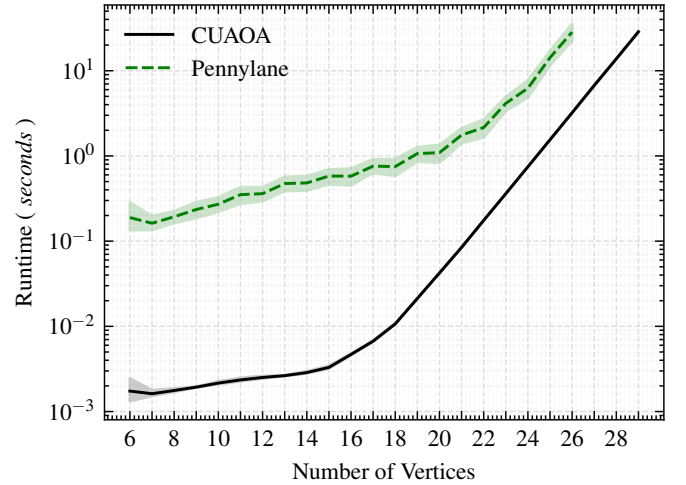


Fig. 3: Runtime for gradient calculation using the adjoint differentiation method with $p = 6$.

Lastly, we also provide an implementation of the gradient based version of the optimizers L-BFGS (natively) [29] and BFGS (through `scipy`) [30]. Additional experiments (not displayed here for brevity) show, that CUAOA is still up to two orders of magnitude faster than PennyLane when using the

same optimizer (BFGS), essentially mirroring the results of Fig. 3.

VI. CONCLUSION

In this paper, we proposed a classical high performance CUDA-based QAOA circuit simulator (CUAOA) oriented towards single-GPU usage. By exploiting speedups enabled through the diagonal structure of the cost operator at multiple stages of the QAOA simulation, i.e., (1) the computation and application of the cost operator, (2) the computation of the expectation value, and (3), providing a QAOA-specialized gradient computation method based on adjoint differentiation, our proposed implementation of the CUAOA outperformed the state-of-the-art QAOA simulator QOKit by an order of magnitude (i.e., a 10-fold speedup) for small to medium sized problem instances. For large scale problem instances above 20 qubits, our approach also performed better than QOKit but equally suffers from the dominating runtime of the mixer operator. Notably, CUAOA offers significantly more functionality than QOKit for the key applications of (1) sampling from the statevector and (2) a GPU-based gradient computation. Further, our gradient computation runs about two orders of magnitude faster than the respective state-of-the-art approach. In conclusion, our approach can be regarded as the new state of the art for single-GPU QAOA simulation, as it outperformed all baselines up to multiple orders of magnitude in a representative evaluation.

In future work, our approach could be extended towards multi-GPU scenarios, which would require mostly additional implementation while relying on the same theoretical insights. A core benefit of this would be the possibility to massively parallelize large amounts of QAOA circuit simulations, which is a typical task in benchmarking quantum optimization. Further, one could natively implement constraint-preserving mixers, by directly reducing the search space, which could significantly reduce numerical simulation runtime for heavily constrained problems.

ACKNOWLEDGMENT

This paper was partially funded by the German Federal Ministry for Economic Affairs and Climate Action through the funding program "Quantum Computing – Applications for the industry" based on the allowance "Development of digital technologies" (contract number: 01MQ22008A) and through the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus.

REFERENCES

- [1] J. Blenninger, J. Stein, D. Bucher, P. J. Eder, E. Çetiner *et al.*, "CUAOA: A Novel CUDA-Accelerated Simulation Framework for the QAOA," Jul. 2024.
- [2] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," 2014.
- [3] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, "Quantum computation by adiabatic evolution," 2000.
- [4] M. Born and V. Fock, "Beweis des Adiabatsatzes," *Zeitschrift für Phys.*, vol. 51, no. 3, pp. 165–180, 1928.
- [5] S. Boulebnane and A. Montanaro, "Solving boolean satisfiability problems with the quantum approximate optimization algorithm," 2022.
- [6] R. Shaydulin, C. Li, S. Chakrabarti, M. DeCross, D. Herman *et al.*, "Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem," *Science Advances*, vol. 10, no. 22, p. eadm6761, 2024.
- [7] C. Carlson, Z. Jorquera, A. Kolla, and S. Kordonowy, "A quantum advantage over classical for local max cut," 2023.
- [8] S. H. Sack and D. J. Egger, "Large-scale quantum approximate optimization on nonplanar graphs with machine learning noise mitigation," *Phys. Rev. Res.*, vol. 6, p. 013223, Mar 2024.
- [9] M. Medvidović and G. Carleo, "Classical variational simulation of the Quantum Approximate Optimization Algorithm," *npj Quantum Inf.*, vol. 7, no. 1, p. 101, 2021.
- [10] H. Shang, Y. Fan, L. Shen, C. Guo, J. Liu *et al.*, "Towards practical and massively parallel quantum computing emulation for quantum chemistry," *npj Quantum Inf.*, vol. 9, no. 1, p. 33, 2023.
- [11] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa *et al.*, "cuquantum sdk: A high-performance library for accelerating quantum science," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 1050–1061.
- [12] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman *et al.*, "Quantum computing with Qiskit," 2024.
- [13] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022.
- [14] NVIDIA Corporation, *NVIDIA CUDA Compute Unified Device Architecture Programming Guide*. NVIDIA Corporation, 2007.
- [15] D. Lykov, R. Shaydulin, Y. Sun, Y. Alexeev, and M. Pistoia, "Fast simulation of high-depth qaoa circuits," in *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1443–1451.
- [16] Fino and Algazi, "Unified matrix treatment of the fast walsh-hadamard transform," *IEEE Transactions on Computers*, vol. C-25, no. 11, pp. 1142–1146, 1976.
- [17] A. Bärttschi and S. Eidenbenz, "Grover mixers for qaoa: Shifting complexity from mixer design to state preparation," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2020, pp. 72–82.
- [18] D. Lykov, R. Shaydulin, Y. Sun, Y. Alexeev, and M. Pistoia, "GitHub repository QOKit," <https://github.com/jpmorganchase/QOKit>, 2024.
- [19] PyO3 Project and Contributors, "PyO3,"
- [20] S. H. Sack and M. Serbyn, "Quantum annealing initialization of the quantum approximate optimization algorithm," *Quantum*, vol. 5, p. 491, Jul. 2021.
- [21] T. Jones and J. Gacon, "Efficient calculation of gradients in classical simulations of variational quantum algorithms," 2020.
- [22] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, "Quantum circuit learning," *Phys. Rev. A*, vol. 98, p. 032309, Sep 2018.
- [23] NVIDIA Corporation, "NVIDIA CUDA C programming guide," 2021.
- [24] V. Sharma, N. S. B. Saharan, S.-H. Chiew, E. I. R. Chiacchio, L. Disilvestro *et al.*, "Openqaoa – an sdk for qaoa," 2022.
- [25] J. Golden, A. Baertschi, D. O'Malley, E. Pelofske, and S. Eidenbenz, "Juliqaoa: Fast, flexible qaoa simulation," in *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W 2023. ACM, Nov. 2023.
- [26] R. L. Graham, T. S. Woodall, and J. M. Squyres, "Open mpi: A flexible high performance mpi," in *Parallel Processing and Applied Mathematics*, R. Wyrzykowski, J. Dongarra, N. Meyer, and J. Waśniewski, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 228–239.
- [27] P. Erdős and A. Rényi, "On random graphs. I," *Publicationes Mathematicae Debrecen*, vol. 6, no. 3-4, pp. 290–297, Jul 1959.
- [28] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: a llvm-based python jit compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, ser. LLVM '15. New York, NY, USA: Association for Computing Machinery, 2015.
- [29] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Math. Program.*, vol. 45, no. 1, pp. 503–528, 1989.
- [30] J. Nocedal, "Updating quasi newton matrices with limited storage," *Mathematics of Computation*, vol. 35, no. 151, pp. 951–958, Jul. 1980.